Taller Archivos

Universidad del Quindio

Facultad de Ingeniería

Programa de Ingeniería en Sistemas y Computación

Christian David Lopera Castaño
Juan Manuel Castaño Buitrago
Samuel Mejia Guerrero

Presentado a:
Carlos Andrés Flórez Villarraga

1/11/2026

Armenia, Quindio

TALLER: ARCHIVOS

Cada grupo entrega un documento PDF con todos los resultados solicitados y los archivos fuente `.exs` listos para ejecutar. La estructura del código debe seguir lo visto en clase y aplicar buenas prácticas de desarrollo.

Se permite el uso de inteligencia artificial como apoyo. Sin embargo, la nota dependerá en buena parte de la comprensión que el grupo demuestre en el documento entregado, más que del código implementado.

Parte 1. Ejercicios

Ejercicio 1. Frecuencia de palabras

Cree un programa que lea un archivo de texto y genere dos archivos.

- **texto_normalizado.txt** con el mismo texto en minúsculas y sin signos de puntuación, una línea por cada línea original. En este ejercicio el archivo de entrada puede leerse completo en memoria.
- **frecuencia.txt** con la frecuencia de cada palabra, calculada a partir de texto_normalizado.txt y ordenada de mayor a menor. Cada línea tiene la forma `palabra: frecuencia`.

El programa debe funcionar correctamente con textos que contengan tildes, eñes, signos de apertura (¿ ¡), espacios repetidos, tabuladores y líneas vacías. Pruébelo al menos con este texto e incluya la salida obtenida en el PDF.

```
¡Hola, José! ¿Cómo está?
El niño comió piña... y PIÑA otra vez.

Árbol,  árbol y más ÁRBOLES en Armenia.
El co-autor llegó el 29/10/2026.
```

Los signos de puntuación se reemplazan por un espacio. Por ejemplo, “co-autor” produce las palabras co y autor, y “29/10/2026” produce 29, 10 y 2026. Los números se cuentan como palabras. Las palabras con la misma frecuencia se ordenan alfabéticamente.

Esta es la salida con las reglas definidas para el ejemplo dado:

texto_normalizado.txt

```
hola josé cómo está
el niño comió piña y piña otra vez

árbol árbol y más árboles en armenia
el co autor llegó el 29 10 2026
```

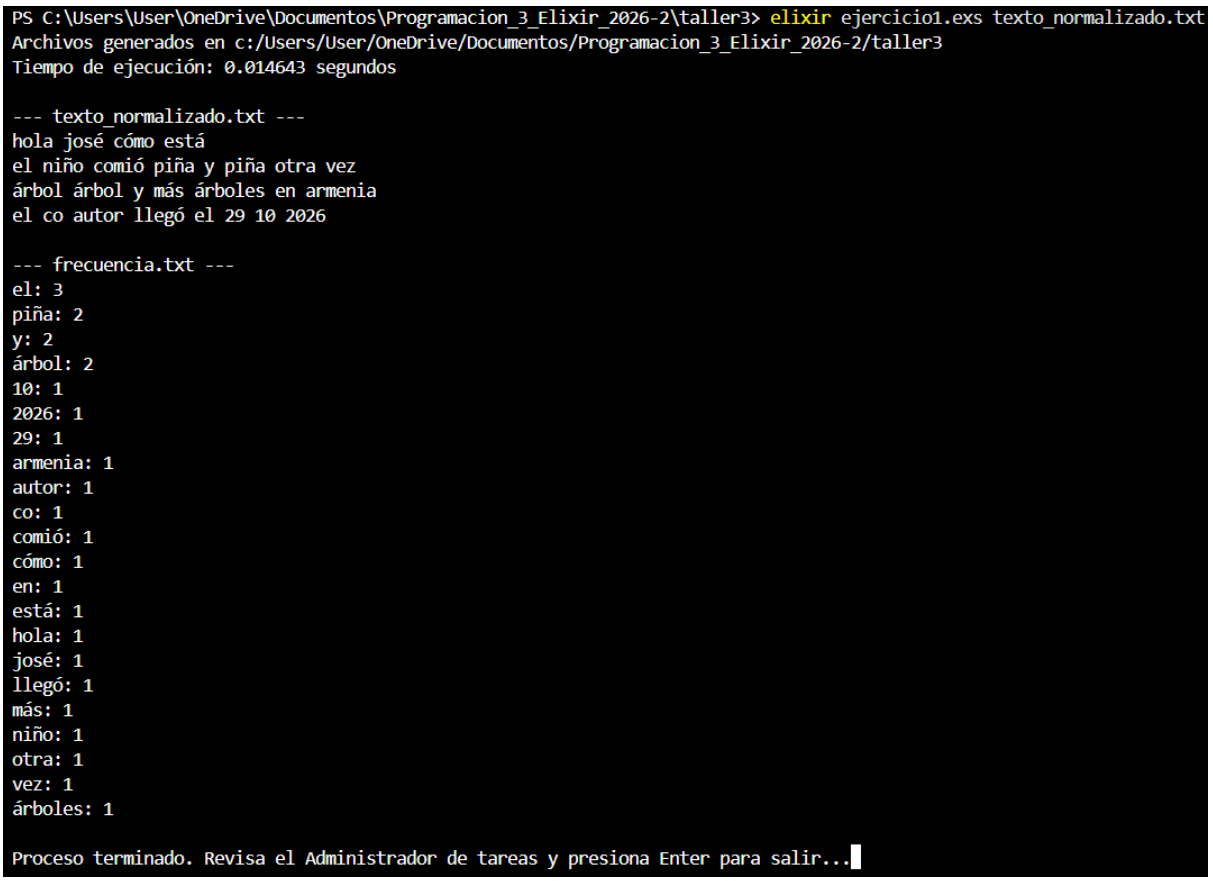
frecuencia.txt

```
el: 3
piña: 2
y: 2
árbol: 2
10: 1
2026: 1
29: 1
armenia: 1
autor: 1
co: 1
comió: 1
cómo: 1
en: 1
está: 1
hola: 1
josé: 1
llegó: 1
más: 1
niño: 1
otra: 1
vez: 1
árboles: 1
```

```
=== Ejercicio 1: frecuencia de palabras (todo en memoria) ===
Archivo de entrada (Enter para usar texto.txt):
Carpeta de salida (Enter para usar la carpeta del programa):
Archivos generados en c:/Users/User/OneDrive/Documentos/Programacion_3_Elixir_2026-2/taller3
Tiempo de ejecución: 0.133017 segundos

--- texto_normalizado.txt ---
hola josé cómo está
el niño comió piña y piña otra vez
árbol árbol y más árboles en armenia
el co autor llegó el 29 10 2026

--- frecuencia.txt ---
el: 3
piña: 2
y: 2
árbol: 2
10: 1
2026: 1
29: 1
armenia: 1
autor: 1
co: 1
comió: 1
cómo: 1
en: 1
está: 1
hola: 1
josé: 1
llegó: 1
más: 1
niño: 1
otra: 1
vez: 1
árboles: 1
```



Ejercicio 2. Procesamiento de un CSV

Dado el archivo **productos.csv**

```
nombre,precio,cantidad
Laptop,1200,10
Mouse,25,500
Teclado,75,30
Monitor,300,15
```

cree un módulo que lea el archivo, convierta cada registro en un mapa con las claves nombre, precio y cantidad, y genere un reporte en inventario.txt con el valor total del inventario (suma de precio por cantidad), el producto más caro, el producto más barato y la lista de productos ordenados de mayor a menor valor en inventario.

Además, el programa debe cumplir estas condiciones.

- Un nombre puede contener comas si está entre comillas dobles, por ejemplo "Monitor 27 pulgadas, curvo",300,15.
- Los precios pueden tener decimales.
- Las líneas inválidas (campos faltantes, valores no numéricos, precios negativos, nombre vacío) no detienen el programa. Se registran en errores.txt con el número de línea y el motivo.

- Si el archivo no existe o no contiene productos válidos, el programa informa la situación sin terminar con una excepción.

Construya su propio archivo de prueba con al menos seis líneas problemáticas distintas e inclúyalo en la entrega.

Probado con el archivo **productos.csv** dado tal cual:

```
PS C:\Users\User\OneDrive\Documentos\Programacion_3_Elixir_2026-2\taller3> elixir ejercicio2.exs
=== Ejercicio 2: inventario de productos ===
Archivo CSV (Enter para usar productos.csv):
Carpeta de salida (Enter para usar la carpeta del programa, o escriba por ejemplo salida_prueba):
Productos válidos: 4. Líneas con error: 0. Archivos generados en c:/Users/User/OneDrive/Documentos/Programacion_3_Elixir_2026-2/taller3

--- inventario.txt ---
=== Reporte de inventario ===
Valor total del inventario: 31250.00
Producto más caro: Laptop (1200.00)
Producto más barato: Mouse (25.00)

Productos ordenados por valor en inventario (mayor a menor):
1. Mouse: 12500.00 (25.00 x 500)
2. Laptop: 12000.00 (1200.00 x 10)
3. Monitor: 4500.00 (300.00 x 15)
4. Teclado: 2250.00 (75.00 x 30)

--- errores.txt ---
No se encontraron líneas inválidas.
```

Probado con el archivo **productos_prueba.csv** con lineas cambiadas, quedando asi:

```
PS C:\Users\User\OneDrive\Documentos\Programacion_3_Elixir_2026-2\taller3> elixir ejercicio2.exs productos_prueba.csv
Productos válidos: 5. Líneas con error: 6. Archivos generados en c:/Users/User/OneDrive/Documentos/Programacion_3_Elixir_2026-2/taller3

--- inventario.txt ---
=== Reporte de inventario ===
Valor total del inventario: 35750.00
Producto más caro: Laptop (1200.00)
Producto más barato: Mouse (25.00)

Productos ordenados por valor en inventario (mayor a menor):
1. Mouse: 12500.00 (25.00 x 500)
2. Laptop: 12000.00 (1200.00 x 10)
3. Monitor: 4500.00 (300.00 x 15)
4. Monitor 27 pulgadas, curvo: 4500.00 (300.00 x 15)
5. Teclado: 2250.00 (75.00 x 30)

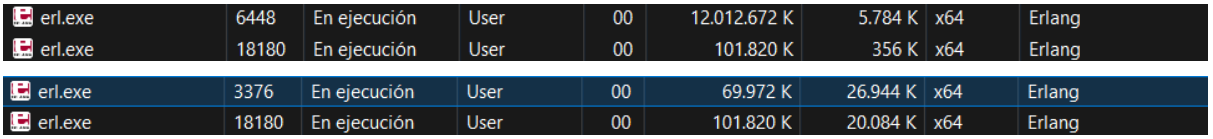
--- errores.txt ---
Línea 7: se esperaban 3 campos y hay 2
Línea 8: el precio no es un número ("veinticinco")
Línea 9: el precio es negativo (-300)
Línea 10: el nombre está vacío
Línea 11: la cantidad no es un número entero ("diez")
Línea 12: la cantidad no es un número entero ("")
```

Parte 2. Experimento con un archivo grande

Escriba una segunda versión del ejercicio 1 que procese el archivo línea por línea con streams, sin cargarlo completo en memoria, tanto en la normalización como en el conteo de palabras. Ambas versiones deben producir exactamente los mismos archivos de salida.

Genere un archivo de texto de al menos 1 GB y ejecute las dos versiones sobre él. Mida el tiempo de ejecución (con `:timer.tc/1`) y la memoria máxima usada de cada una (investigue cómo se obtiene este valor). En el PDF presente los resultados en una tabla y explique con sus propias palabras a qué se deben las diferencias.

Estos fueron los resultados que me dieron, y para medir la memoria maxima usada, use la parte de detalles en el administrador de tareas y agregue la columna de espacio maximo de trabajo, y para que la tarea no se desapareciera, añadi una linea que dijera que para seguir habia que darle enter y poder medir la memoria maxima usada, y estos fueron los resultados



erl.exe	6448	En ejecución	User	00	12.012.672 K	5.784 K	x64	Erlang
erl.exe	18180	En ejecución	User	00	101.820 K	356 K	x64	Erlang

erl.exe	3376	En ejecución	User	00	69.972 K	26.944 K	x64	Erlang
erl.exe	18180	En ejecución	User	00	101.820 K	20.084 K	x64	Erlang

Versión	Tiempo de Ejecución	Memoria Máxima Usada (RAM)
V1: Memoria Completa (`File.read`)	852.28 segundos (14.2 min)	12 GB (12.012.672 K)
V2: Streams (`File.stream!`)	330.40 segundos (~5.5 min)	101.8 MB (101.820 K)

La versión 1 carga todo el archivo de golpe y crea copias de datos en cada paso (`String.split`, `String.replace`), eso hace que se consuman 12 GB. Esto obliga al sistema operativo a usar el disco duro como memoria de emergencia, lo que frena el procesamiento (14 minutos), en cambio la version 2, al hacer el proceso línea por línea, solo necesita unos 100 MB de RAM, permitiendo que el procesador trabajara libremente y terminara en solo 5 minutos y medio

Y esta fue la salida por consola que me dio

```
Tiempo de ejecución: 330.403225 segundos

--- texto_normalizado.txt es muy grande para mostrarlo en pantalla ---

--- frecuencia.txt ---
de: 34500000
clase: 11500000
el: 11500000
elixir: 11500000
es: 11500000
este: 11500000
evaluando: 11500000
la: 11500000
para: 11500000
programacion: 11500000
prueba: 11500000
rendimiento: 11500000
texto: 11500000
un: 11500000

Proceso terminado. Revisa el Administrador de tareas y presiona Enter para salir...
PS C:\Users\User\OneDrive\Documentos\Programacion_3_Elixir_2026-2\taller3> elixir ejercicio1.exs archivo_1gb.txt
Archivos generados en c:/Users/User/OneDrive/Documentos/Programacion_3_Elixir_2026-2/taller3
Tiempo de ejecución: 852.286309 segundos

--- texto_normalizado.txt es muy grande para mostrarlo en pantalla ---

--- frecuencia.txt ---
de: 34500000
clase: 11500000
el: 11500000
elixir: 11500000
es: 11500000
este: 11500000
```

```
este: 11500000
evaluando: 11500000
la: 11500000
para: 11500000
programacion: 11500000
prueba: 11500000
rendimiento: 11500000
texto: 11500000
un: 11500000

Proceso terminado. Revisa el Administrador de tareas y presiona Enter para salir...
PS C:\Users\User\OneDrive\Documentos\Programacion_3_Elixir_2026-2\taller3>
```

Parte 3. Bitácora de uso de IA

Incluya en el PDF entre tres y cinco consultas que el grupo hizo a una herramienta de IA durante el taller. Para cada una indique qué preguntaron, qué les sugirió la herramienta y qué decidieron hacer con esa sugerencia. Señale qué conceptos o funciones no conocían antes y qué aprendieron de ellos.

¿Qué preguntaron?	¿Qué les sugirió?	¿Qué decidieron hacer?
Preguntamos cómo utilizar la función `:timer.tc/1` para medir cuánto tiempo tarda en ejecutarse una función en Elixir.	La herramienta nos explicó que `:timer.tc/1` permite medir el tiempo de ejecución de una función y devuelve una tupla con el tiempo transcurrido en microsegundos y el resultado de la función.	Decidimos utilizar esta función para medir el tiempo que tardan en ejecutarse las dos versiones que el profesor nos pidió.
Preguntamos cómo investigar y obtener el valor de la memoria máxima utilizada por el programa al procesar el archivo de 1 GB para incluirlo en la tabla comparativa del informe.	La herramienta sugirió utilizar el Administrador de tareas de Windows, ingresando a la pestaña *Detalles*, activando la columna "Pico de trabajo de memoria" (Peak working set) y rastreando el proceso `erl.exe` de la máquina virtual de Erlang durante la ejecución de cada versión.	Decidimos seguir esta guía para capturar de forma visual y confiable el consumo real de RAM (observando el contraste entre los picos de la versión en memoria frente a la versión con *Streams*), obteniendo la evidencia empírica exacta para nuestra tabla.
Preguntamos cómo utilizar Streams en Elixir para procesar un archivo de texto línea por línea, sin necesidad de cargar todo su contenido en la memoria.	La herramienta nos sugirió utilizar `File.stream!/1`, que permite leer un archivo progresivamente y procesar cada línea	Decidimos implementar una segunda versión del ejercicio utilizando file.streams
Preguntamos cómo leer el contenido de un archivo de texto utilizando Elixir y cómo almacenar esa información para poder trabajar con ella.	La herramienta nos explicó que podíamos utilizar `File.read/1`, una función que permite leer el contenido completo de un archivo y obtenerlo como una cadena de texto.	Decidimos utilizar `File.read/1` para leer el archivo completo y trabajar con su contenido en memoria, ya que esta opción nos permitía desarrollar la primera versión del ejercicio.

¿Qué conceptos o funciones no conocíamos?

Antes de hacer el taller no conociamos la funcion `:timer.tc/1 al igual que como medir la memoria maxima usada ni las dos funciones que tuvimos que implementar para hacer la version 2 que eran: File.read/1 y File.stream!/1.`

¿Qué aprendimos?

Aprendimos que `:timer.tc/1` es una función del módulo Timer de Erlang que podemos utilizar directamente en Elixir. Antes no conocíamos esta función ni sabíamos que era posible medir el tiempo de ejecución de una función de esta manera.

Aprendimos a utilizar `File.read/1` para leer archivos en Elixir y conocimos que esta función carga todo el contenido del archivo en memoria. Antes no conocíamos esta función ni su funcionamiento.

Antes no conocíamos `File.stream!/1` ni la diferencia entre procesar un archivo completo y hacerlo línea por línea.